Hardware Design of Intrusion Detection System for Automotive CAN Bus Using Random Forest

Daegi Lee
Soongsil University
Republic of Korea
dleorl9083@gmail.com

Changseon Han
Soongsil University
Seoul, Korea
ckdtjs0497@naver.com

Seongsoo Lee
Soongsil University
Seoul, Korea
sslee@ssu.ac.kr

2023 International Conference on Electronics, Information, and Communication (ICEIC) | 979-8-3503-2021-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICEIC57457.2023.10049883

***Abstract*—Controller area network (CAN) bus is a reliable protocol that connects electronic control units in a vehicle to send and receive control data for driving a vehicle. However, CAN-bus is quite weak against external attacks since it was not designed considering cybersecurity. In this paper, we propose an intrusion detection system (IDS). It monitors data frames on CAN bus, and detects spoofing attack using random forest, one of the machine learning techniques. When combined with node exclusion system (NES) in the previous works, IDS can effectively detect the hacked node and NES can exclude it from the CAN bus. The proposed IDS and NES have been designed and verified with Modelsim simulator.**

Keywords—Controller Area Network (CAN), Hacking, Cybersecurity, Intrusion Detection System (IDS), Node Exclusion System (NES), Random Forest, Machine Learning

I. INTRODUCTION

The number of connected cars based on V2X communication is increasing, and the number is expected to increase to 775 million by 2023 [1]. As the number of connected cars increases, security threats are also increasing, and stability of communication between electronic components is required. Following these technological trends, the International Telecommunication Union Telecommunication Standardization Sector (ITU-T) defines security issues and threats in the X.itssec standard series, starting with the vehicle's software update guidelines and the V2X guidelines, of which X.itssec-4 defines threats and characteristics of the vehicle's internal network [2]. As such, the need for vehicle security is increasing, and research is being conducted to protect the safety of drivers from attackers.

Controller area network (CAN) buses are most frequently used for in-vehicle networks (IVNs). Through the CAN communication protocol, electronic control units (ECUs) in the vehicle that are closely related to the user's safety are connected. However, since nodes connected to the CAN bus do not have an address and all transmissions are broadcast, it is difficult to identify which node is the problem even if one of the nodes participating in the communication is hacked and sends a malicious data frame. Malicious data frames can also overload the CAN bus, preventing normal communication, or causing unintended frame transmission to act as a risk to the user. Therefore, there is a need for a way to compensate for the security vulnerabilities of the CAN protocol and to detect and cope with malicious frames.

This paper was supported by Industrial Technology Challenge Track (20012624) of the Ministry of Trade, Industry and Energy (MOTIE) / Korea Evaluation Institute of Industrial Technology (KEIT). It was also supported by the R&D Program of the Ministry of Trade, Industry, and Energy (MOTIE) (RS-2022-00154973) and Korea Evaluation Institute of Industrial Technology (KEIT).

In this paper, we use random forest [3] to detect malicious frames at the hardware level. In addition, Modelsim simulator verifies the behavior of node expulsion system (NES) [4]-[6], a system that can guarantee security by assigning unique numbers to each CAN node and excluding hacked ECUs from CAN bus.

II. CLASSIFIER AND ATTACK

A. Random Forest

Random forest is a machine learning classifier generated by an ensemble learning algorithm. Using the bootstrap method, samples of a dataset are randomly extracted to generate several small-sized decision trees, and the class probability values of these trees are averaged to predict the class with the highest probability. Random forest has the advantage of less overfitting training datasets in that it uses several bootstrap samples and small decision trees, and is evaluated as a good classifier in that it finally checks the class prediction once more. In addition, the ensemble model Random Forest is very effective when classifying classes for structured data such as CAN frame [7]. The random forest proposed in this paper is designed by synthesizing decision trees trained through the WEKA (Waikato Environment for Knowledge Analysis) program [8], a public-domain machine learning tool set.

B. Spoofing Attack

It is showed that automobile control is possible by accessing IVN and injecting CAN messages [9]. This attack may overload the CAN bus or cause unintended action, posing a threat to the driver's safety.

In this paper, we only deal with spoofing attacks in which an attacker hacks an ECU and pretends to be a normal ECU and transmits a CAN Message [10]. Fig. 1 shows an example of three ECUs connected to the CAN bus, of which ECU3 was occupied by an attacker. In this example, the ECU3 transmits the CAN frame by mimicking the message ID 0x012 of the ECU1. Fig. 2 shows a phenomenon that damages the periodicity of a frame with an ID of 0x012 by transmitting a frame imitated by ECU3. Through this attack method, the driver may induce unintended movement.

C. Feature

The feature for training the decision tree is extracted through a Dataset that contains the transmitted time and information of the frame. We use the anomaly method to calculate the average Feature value of each CAN ID in a normal dataset that does not include an attack and compare it with the Feature calculated in the attack dataset [11]. This

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 11,2026 at 07:28:25 UTC from IEEE Xplore. Restrictions apply.

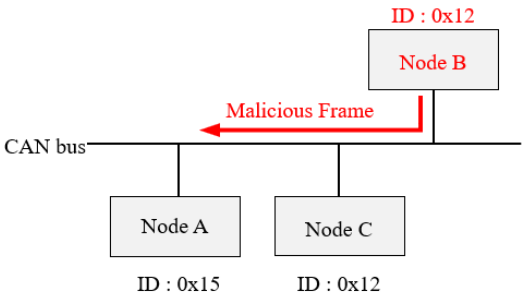


Fig. 1 Example of Attacked Node in CAN Bus

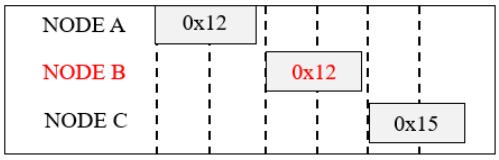


Fig. 2 Spoofing Attack in CAN Transmission

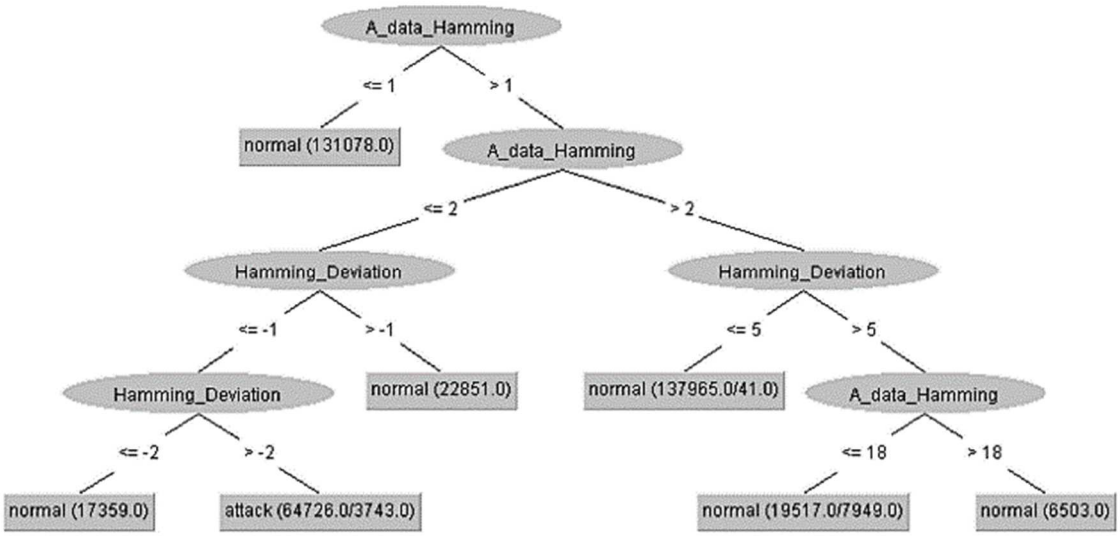


Fig. 3 Example of WEKA Model Visualization

paper uses the frame cycle and feature using payload's Hamming distance.

The CAN Protocol is divided into an aperiodic frame and a periodic frame. As for aperiodic frames, diagnostic frames through OBD-II ports or event frames by trigger exist [12]. In most other cases, periodic frames that transmit data at the same period account for a large proportion. If an attacker injects a spoofing frame into the CAN bus, the period of the frame changes, so the attack is detected using the time difference between frames with the same ID [13]. The time difference between frames can be obtained through the following equation.

$$Interval[i] = Time\ (Msg[i]) - Time(Msg[i-1]) \quad (1)$$

Using (1), the interval average for each ID may be obtained from the normal dataset.

$$Interval\ Sum = \sum_{k=0}^{N-1} N_Interval[k] \quad (2)$$

$$Interval\ Average = \ Interval\ Sum\ /\ N \quad (3)$$

In the above equation, N means the number of frames with the same ID in Dataset, and N_Interval means the interval calculated in the normal dataset. Through the interval average, the deviation from the interval value calculated in the dataset including the spoofing attack can be calculated.

$$Interval\ Deviation = \ Interval[i] - Interval\ Average \quad (4)$$

This deviation value indicates how far the frame period deviates from the normal situation, and it can be determined that the more difference there is, the higher attack potential.

Calculating the Hamming distance between payloads with the same ID is a simple algorithm that can be implemented using only low memory, so it is suitable for ECUs with limited resources [14]. Hamming distance is also calculated through the same procedure as the procedure for calculating Interval.

$$Hamming[i] = Msg[i]_{payload} \oplus Msg[i-1]_{payload} \quad (5)$$

As the difference between the CAN controller's operating speed and the driver's operating speed is large, the payload value slowly changes, and if a frame with payload change outside the normal range is manifested as a deviation value of Hamming Distance, it can be determined that the attack potential is high.

III. Proposed Design

To accelerate the decision tree models that make up Random Forest to quickly detect attacks. The proposed design was designed with reference to the memory-central method[15].

A. WEKA

The decision trees need to be visualized to implement Random Forest in hardware. For this reason, we confirmed the structure of the model generated using WEKA, a machine learning training program. In WEKA, we trained the model using a dataset of the features described above.

The circle represents the feature used by the node, and the square represents the sample class that classified according to the threshold written on the line.

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 11,2026 at 07:28:25 UTC from IEEE Xplore. Restrictions apply.

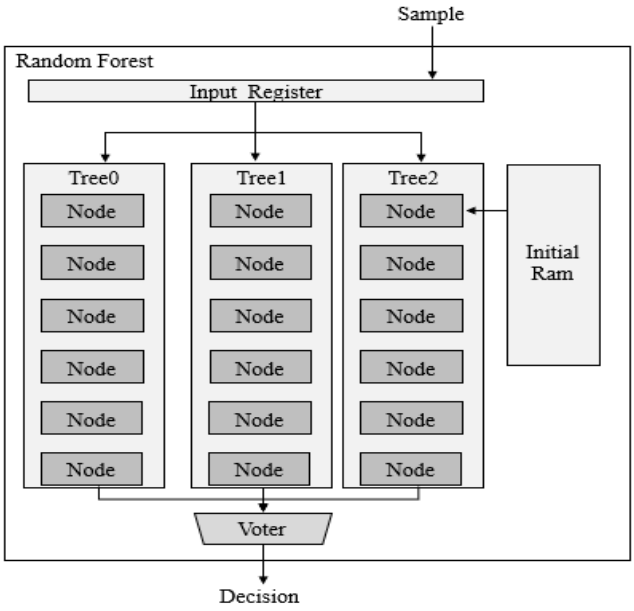


Fig. 4 Random Forest Hardware Design

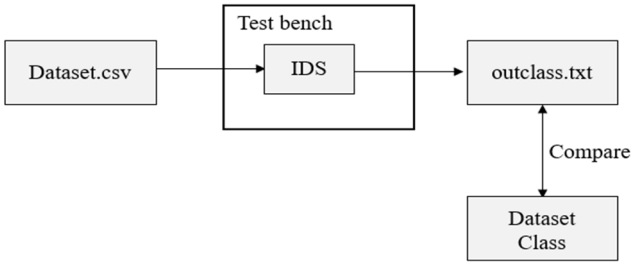


Fig. 5 IDS Simulation Process

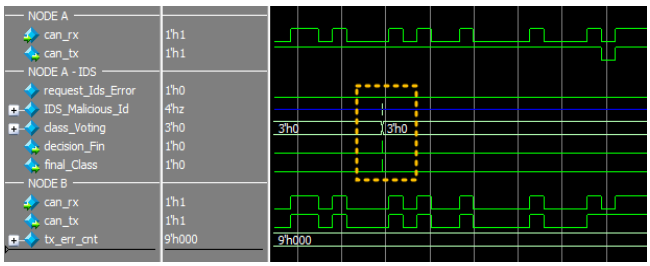


Fig. 6 Interaction of IDS and NES

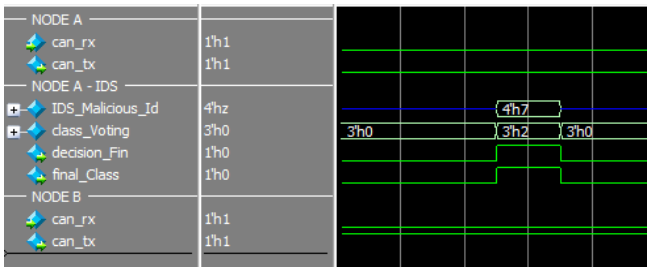


Fig. 7 CAN Node Unique Number Output by Detection

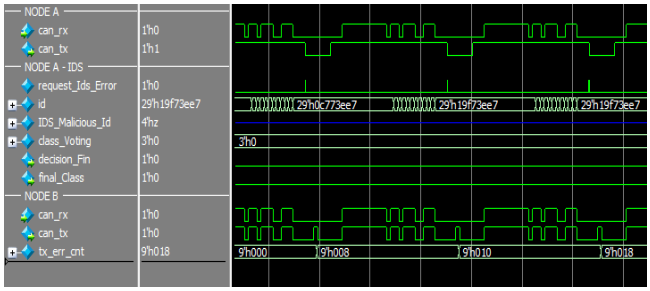


Fig. 8 Actions of IDS and NES against Hacked ECU

B. Hardware Implementation

Fig. 4 shows hardware structure of random forest. In order to determine the class of the frame, the forest level receives a sample, which consists of feature values. Each of the decision tree models constituting the random forest model is input at the same time. The initial RAM inside the model stores parameters that make up each decision tree, as shown in Fig. 3, and when it boots up, it sends the relevant information to the nodes of each tree to set the initial value. The nodes of the tree determine the class for the input sample through the following procedure.

1) Temporarily store the sample's feature value according to the feature used in the node.

2) Compare the threshold and sample feature values stored in the node with each other. WEKA represents a model based on the operation of Feature <= Threshold.

3) Based on the structure of Fig. 3, if the calculation result is true, move to the left, and if the left child node means class determination, the ID and class of the CAN frame are transmitted to voter or sample data is transmitted to the child node.

The parameters of the decision tree model are fixed unless newly trained, so once delivered, the class classification of the feature can be performed while maintaining the value This method has the advantage of being able to complete the classification in a short period of time in 1 cycle after the sample is entered into the node and then moved to the next node or class is classified.

If the class of the input sample is determined, it is finally determined by voter below Fig. 4. In this paper, since class is classified only as attack (=1) or normal (=0), if the results of each tree are combined, the sample is classified as a malicious frame. In order to prevent a tie from occurring in this process, the number of trees shall be an odd number.

IV. Simulation

In this paper, two simulations are conducted. First, in order to verify only the operation of IDS, simulation is performed to calculate accuracy using dataset. Second, simulation is performed to confirm the interaction between NES and IDS.

Fig. 5 shows the process for performing the first simulation. Since the process of receiving the frame of CAN Bus and determining it as IDS is inefficient to test the performance of IDS, IDS directly inputs and classifies dataset's feature, and outputs the result value as a text file. This text file shows the order of frames and classified classes. After that, the accuracy was calculated by comparing the text file and dataset with each other. Through this process, it was confirmed that IDS had 98.2% accuracy, and simulation results were obtained that the number of trees was small, so it was not significantly different from the model with high accuracy among the several decision trees trained by WEKA program.

Next, an interactive operation when the IDS is connected to the NES is confirmed as shown in Fig. 6. This simulation uses only the top 7 bits of arbitration ID because it uses NES, which uses the bottom 4 bits of the frame ID as the unique address of the node. In the case of a spoofing attack by the hacked node B in Fig. 1, Node A including IDS detects a malicious frame and outputs a detection signal. Fig. 7 is an enlarged simulation screen of the orange square in Fig. 6. In

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 11,2026 at 07:28:25 UTC from IEEE Xplore. Restrictions apply.

IDS's voter, two of the three trees classified as attack are more than half, and the final input sample is judged as malicious frame. In addition, the IDS detects the malicious frame and outputs 0x07 which was the unique node number of node B. It was confirmed through simulation that if a message ID with the same node number appears after storing this number in an internal buffer, it can be dealt with by increasing the transmission error count of the transmission node as shown in Fig. 8 by NES.

V. Conclusion

This paper proposes a hardware IDS optimized for detection of spoofing attacks in CAN networks. IDS using Random Forest was implemented by modeling the decision tree with the WEKA program and designing and synthesizing the trees with Verilog HDL. The CAN protocol was vulnerable to security in that the sender was unknown and all transmissions were broadcast. Since the spoofing attack using this point is an attack that injects a message into the CAN Bus by pretending to be normal, the deviation from the Payload Hamming distance using the difference in control speed and the average that can detect sudden changes was used. Using these features, it was possible to detect malicious frames used in the spoofing attack injected into the CAN Bus. We also validate via Behavior Simulation that NES can identify and respond to which ECUs were attacked to secure security vulnerabilities in the CAN protocol.

References

[1] Upstream Security, “Upstream’s 2022 Global Automotive Cybersecurity Report”, 2022

[2] C. Kim, “A Study on the Standard Development Trend for Automotive Security Threats”, Review of KIISC, vol. 29, no. 1, pp. 20-25, 2019.

[3] L. Breiman, “Random Forests”, Machine Learning, vol. 45, 2001.

[4] T. Kang, “Error Detection and Correction Techniques for CAN Bus and SoC implementation”, Master Thesis, Soongsil University, 2019.

[5] T. Kang and S. Lee, “Counterattack Method against Hacked Node in CAN Bus Physical Layer”, j.inst.Korean.electr.electron.eng., vol. 23, no. 4, pp. 1469-1472, 2019.

[6] T. Kang, J. Lee, and S. Lee, “Physical Layer Security Method with CAN Bus Node ID Auto-Setting”, j.inst.Korean.electr.electron.eng., vol. 24, no. 1, pp. 665-668, 2020.

[7] H. Park, Self-Studying Machine Learning + Deep learning, Hanbit Media, 2020.

[8] Waikato Environment for Knowledge Analysis (WEKA), https://www.weka.io

[9] C. Miller and C. Valasek, “Adventures in Automotive Networks and C ontrol Units”, https://ioactive.com/pdfs/IOActive_Adventures_in_Aut omotive_Networks_and_Control_Units.pdf

[10] E. Seo, H. Song, and H. Kim, “GIDS: GAN-Based Intrusion Detection System for In-Vehicle Network.” Proceesings of Annual Conference on Privacy, Security and Trust, pp. 1-6, 2018.

[11] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, “Anomaly-Based Network Intrusion Detection: Techniques, Systems and Challenges”, Computers & Security, vol. 28, no. 1-2, pp. 18-28, 2009.

[12] T. Kim, B. Cho, and J. Choi, “CAN Bus Monitoring for Improving Vehicle Safety”, Proceedings of KSAE Annual Spring, pp. 671-672, 2017.

[13] H. Song, H. Kim, and H. Kim, “Intrusion Detection System-Based on the Analysis of Time Intervals of CAN Messages for In-Vehicle Network”, Proceedings of International Conference on Information Networking , pp. 63-68, 2016.

[14] D. Stabili, M. Marchetti, and M. Colajanni, "Detecting Attacks to Internal Vehicle Networks through Hamming Distance," Proceedings of AEIT International Annual Conference, pp. 1-6, 2017.

[15] X. Lin, R. Blanton, and D. Thomas, “Random Forest Architectures on FPGA for Multiple Applications”, Proceedings of GLS-VLSI, p. 415-418, 2017.

Authorized licensed use limited to: Notre Dame University - Louaize. Downloaded on September 11,2026 at 07:28:25 UTC from IEEE Xplore. Restrictions apply.